

Дизайн и анализ на алгоритми упражнение 7

Станислав Игов (Стъката)

2025/2026

Алчни алгоритми. Minimum Arrows. Assign Cookies. Недоминирани точки.

Задача 1 (Minimum Number of Arrows to Burst Balloons)

Радо се прибрал на село за *Великденските празници*, където както всяка година имаше весел карнавал на площада. Сред сергиите с козунаци и боядисани яйца, най-много го привлече една игра с балони, залепени по голяма стена.

Всеки балон заемаше определено място по хоризонтала — някои бяха по-широки, други по-тесни, а някои дори се припокриваха. Стопанинът на играта обясни правилата: Радо трябва да стреля вертикално нагоре с лък и стрели, като всяка стрела минава по права линия и пука всички балони, които срещне по пътя си.

Целта на Радо е не просто да спуска всички балони, а да го направи възможно най-умно — с **минимален брой стрели**. Той започна да оглежда внимателно разположението на балоните и да мисли къде да насочи всяка стрела, така че да улучи колкото се може повече от тях наведнъж.

Помогнете на Радо, как да избере най-добрите позиции за стрелба, така че всички балони да бъдат спукани с най-малко усилие.

Формализация на задачата:

Нека $P[1 \dots n]$ е масив от двойки $(x_{\text{start}}, x_{\text{end}})$, описващи хоризонталните диаметри на балоните.

Балон i заема интервала $[x_{\text{start}}^i, x_{\text{end}}^i]$ по оста x .

Ако Радо изстреля стрела вертикално от позиция x , тя ще спуска всички балони i , за които е изпълнено:

$$x_{\text{start}}^i \leq x \leq x_{\text{end}}^i.$$

Една стрела може да спуска няколко балона, ако техните интервали се припокриват в избраната точка.

Задача: Да се намери минималният брой стрели, които Радо трябва да изстреля, за да спуска всички балони.

Вход: $P[1 \dots n]$: масив от двойки $(x_{\text{start}}, x_{\text{end}})$, описващи хоризонталните диаметри на балони.

Изход: Минималният брой стрели, необходими за да бъдат спукани всички балони.

Балоните са залепени на стена (XY-равнина). Балон i заема хоризонталния интервал $[x_{\text{start}}^i, x_{\text{end}}^i]$. Стрела, изстреляна вертикално от позиция x , спуска балон i тогава и само тогава, когато $x_{\text{start}}^i \leq x \leq x_{\text{end}}^i$. Една стрела може да спуска всички балони по пътя си. Търсим минималния брой стрели.

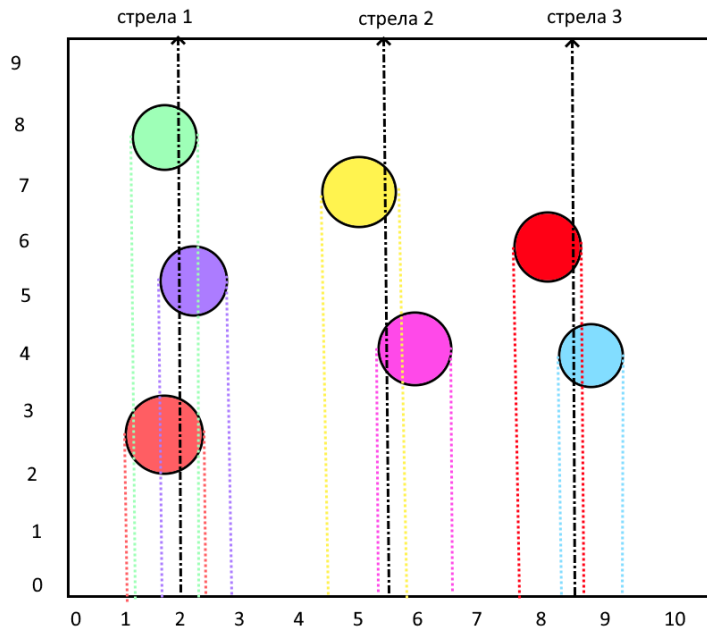
Връзка с Interval Scheduling

Задачата е еквивалентна на следния проблем: дадени са n интервала, намери минимален брой точки, такива че всеки интервал съдържа поне една от тях. Това е задачата **Interval Piercing** (или **Hitting Set** за интервали).

Лема (Оптималност на алчния избор)

Твърдение: Нека интервалите са сортирани по краен момент в нарастващ ред. Тогава съществува оптимално решение, в което първата стрела е изстреляна в крайната точка на първия интервал.

Доказателство (Exchange Argument):



Фигура 1: Примерна картинка за яснота на задачата

Нека OPT е оптимално решение и нека a е стрелата в OPT , която покрива първия интервал $[s_1, e_1]$ (интервалът с най-ранен край). Следователно $s_1 \leq a \leq e_1$.

Конструираме OPT' , като заменяме a с e_1 .

Всеки балон, спукан от a в OPT , има интервал $[s_i, e_i]$ с $s_i \leq a \leq e_i$. Тъй като $a \leq e_1$, имаме $s_i \leq e_1$. Освен това $e_i \geq e_1$ (защото e_1 е минималният краен момент). Следователно $s_i \leq e_1 \leq e_i$ и балонът е спукан и от стрела на позиция e_1 .

Тоест OPT' спуква всички балони, които OPT спуква, със същия брой стрели. Следователно OPT' е оптимално и първата стрела е на позиция e_1 .

Алгоритъм

```
BurstBalloons(P[1..n] : array of pairs)
@1  Сортирай P по x_end нарастващо
@2  arrows <- 0
@3  last_arrow <- -inf
@4  for i = 1 to n do
@5      if P[i].x_start > last_arrow then
@6          arrows <- arrows + 1
@7          last_arrow <- P[i].x_end
@8      endif
@9  endfor
@10 return arrows
```

Алчна идея: Сортираме балоните по краен момент. Обхождаме ги в този ред. Ако текущият балон не е покрит от последната стрела, изстрелваме нова стрела в крайната точка на текущия балон.

Доказателство за коректност (Exchange argument)

Твърдение: BurstBalloons връща минималния брой стрели, необходими за спукване на всички балони.

Нека интервалите са сортирани по краен момент: $e_1 \leq e_2 \leq \dots \leq e_n$.

Нека $G = (g_1, g_2, \dots, g_p)$ са позициите на стрелите на алчния и $OPT = (o_1, o_2, \dots, o_q)$ — на произволно оптимално решение и двете подредени в нарастващ ред.

Ще покажем, че можем да трансформираме ОРТ в G, откъдето $p = q$.

Нека допуснем че ОРТ и G се различават и j е **първият** индекс, на който $g_j \neq o_j$. За всяко $i < j$ имаме $g_i = o_i$, следователно множеството непокрити балони преди стъпка j е едно и също в G и ОРТ.

Нека t е балонът с **най-малък краен момент** сред непокритите от първите $j - 1$ стрели. Тъй като t не е покрит: $s_t > g_{j-1} = o_{j-1}$.

По конструкцията на алчния: $g_j = e_t$.

Допускаме че $o_j > g_j$. Ако $o_j > g_j = e_t$, тогава $o_j > e_t$, т.е. стрелата o_j е вдясно от края на балон t . Стрелите $o_1, \dots, o_{j-1}$ (равни на $g_1, \dots, g_{j-1}$) са вляво от s_t . Стрелите $o_{j+1}, \dots, o_q$ са вдясно от $o_j > e_t$. Следователно **никоя стрела в ОРТ не попада в $[s_t, e_t]$** и балон t не е покрит — противоречие с валидността на ОРТ.

Значи е изпълнено че $o_j < g_j$. Следователно $o_j < g_j = e_t$. Конструираме OPT' , като заменяме o_j с $o'_j = g_j = e_t$.

Като следствие от **Лемата** можем да кажем че всеки балон, покрит от o_j , има начало $s \leq o_j < e_t$ и край $e \geq e_t$ (тъй като t е с минимален край сред непокритите и всеки балон, покрит от o_j , е непокрит от първите $j - 1$ стрели). Следователно $s \leq e_t \leq e$ и стрела на позиция e_t покрива същите балони. Допълнително e_t покрива и балон t (и евентуално още).

Тоест OPT' покрива **всички** балони, които ОРТ покрива, със същия брой стрели. Следователно OPT' е оптимално и $o'_j = g_j$ — OPT' съвпада с G на позиции $1, \dots, j$.

Заклучение. Прилагаме аргумента повторно за OPT' . На всяка стъпка увеличаваме броя позиции, на които ОРТ съвпада с G. След краен брой размени ОРТ се трансформира в G. Следователно $G = OPT$ и алчният е оптимален. За да докажем че $p = q$ можем да допуснем че $p > q$. Понеже видяхме че $o_j \leq g_j$ то ще следва че след G е спукал балон който ОРТ не е, което е противоречие.

Времева сложност

- Ред @1: Сортиране — $O(n \log n)$.
- Редове @4–@9: Цикъл с n итерации, всяка $O(1)$ — $O(n)$.

Следователно $\text{Time}(\text{BurstBalloons}) \in O(n \log n)$.

Задача 2 (Assign Cookies)

Вход: $g[1 \dots m]$: масив от “алчност” на деца (минимален размер бисквитка); $s[1 \dots n]$: масив от размери на бисквитки.

Изход: Максималният брой доволни деца.

Всяко дете i ще бъде доволно, само ако получи бисквитка с размер $\geq g[i]$. Всяко дете получава най-много една бисквитка, всяка бисквитка се дава на най-много едно дете. Целта е да максимизираме броя доволни деца.

Алгоритъм

```
AssignCookies(g[1..m] : naturals, s[1..n] : naturals)
@1  Sort(g[1..m])           // сортираме децата по алчност нарастващо
@2  Sort(s[1..n])           // сортираме бисквитките по размер нарастващо
@3  happy <- 0
@4  i <- 1                   // индекс за деца
@5  j <- 1                   // индекс за бисквитки
@6  while i <= m and j <= n do
@7      if s[j] >= g[i] then
@8          happy <- happy + 1
```

```

@8      i <- i + 1
@9      j <- j + 1
@10     else
@11      j <- j + 1
@12     endif
@13 endwhile
@14 return happy

```

Алчна идея: Сортираме и децата, и бисквитките в нарастващ ред. На всяка стъпка опитваме да задоволим най-неалчното (все още недоволно) дете с най-малката (все още неизползвана) бисквитка, която го удовлетворява. Ако текущата бисквитка не стига, преминаваме към следващата (по-голяма).

Доказателство за коректност (Exchange Argument)

Твърдение: `AssignCookies` връща максималния брой доволни деца.

Нека след сортирането $g[1] \leq g[2] \leq \dots \leq g[m]$ и $s[1] \leq s[2] \leq \dots \leq s[n]$.

Нека G е решението на алчния алгоритъм, а OPT е произволно оптимално решение. Ще покажем, че $|G| \geq |OPT|$.

Ако $OPT = G$, сме готови. Допускаме, че $OPT \neq G$.

Разглеждаме OPT като множество от двойки (дете, бисквитка). Нека (c, b) е двойка в OPT , т.е. дете c получава бисквитка b .

Стъпка 1: OPT може да се трансформира да обслужва децата в нарастващ ред по алчност. Допускаме, че в OPT дете c_1 ($g[c_1] \leq g[c_2]$) получава бисквитка b_1 , а дете c_2 получава бисквитка b_2 , като $s[b_1] > s[b_2]$. Тоест по-неалчното дете получава по-голямата бисквитка.

Разменяме: c_1 получава b_2 , а c_2 получава b_1 . Тъй като $s[b_2] \geq g[c_2] \geq g[c_1]$, дете c_1 е все още доволно. Тъй като $s[b_1] > s[b_2] \geq g[c_2]$, дете c_2 е все още доволно. Размяната запазва валидността и броя доволни деца.

С краен брой такива размени можем да приведем OPT във вид, в който на по-неалчно дете се дава по-малка (или равна) бисквитка.

Стъпка 2: Алчният избор е поне толкова добър. Допускаме, че OPT (след преобразуването от Стъпка 1) и G се различават. Нека c е първото дете (в нарастващ ред по алчност), което е обслужено различно.

Алчният дава на c **най-малката бисквитка**, която го удовлетворява. Нека алчният дава бисквитка b_G , а OPT дава бисквитка b_O . Тъй като и двете удовлетворяват c : $s[b_G] \geq g[c]$ и $s[b_O] \geq g[c]$.

По конструкцията на алчния, $s[b_G] \leq s[b_O]$ (алчният избира най-малката подходяща бисквитка).

Конструираме OPT' : заменяме бисквитка b_O с b_G за дете c . Бисквитка b_O (по-голямата) става свободна и може да задоволи всяко дете, което b_G би могла (и дори повече). Следователно $|OPT'| \geq |OPT|$.

OPT' съпада с G на всички деца до c включително. Прилагаме аргумента повторно. След краен брой размени OPT се трансформира в G , без да намалим броя доволни деца.

Следователно $|G| \geq |OPT|$ и алчният е оптимален.

Времева сложност

- Редове @1–@2: Сортиране — $O(m \log m + n \log n)$.
- Редове @6–@13: Цикъл с най-много $m + n$ итерации, всяка $O(1)$ — $O(m + n)$.

Следователно $\text{Time}(\text{AssignCookies}) \in O(m \log m + n \log n)$.

Задача 3 (Недоминирани точки в равнината)

Дефиниция. За точки в R^2 дефинираме релацията “се доминира от”:

$$(a, b) \preceq (c, d) \stackrel{\text{def}}{\iff} a \leq c \wedge b \leq d$$

Казваме, че точка p е **доминирана**, ако съществува различна точка $q \neq p$ в множеството, такава че $p \preceq q$. Точка p е **недоминирана**, ако не е доминирана от никоя друга точка.

Лема (Съществуване на недоминирана точка)

Твърдение: Всяко крайно непразно множество $P \subseteq R^2$ съдържа поне една недоминирана точка.

Доказателство:

Нека $p^* = (x^*, y^*)$ е точката от P с максимална x -координата. Ако има няколко с еднаква максимална x -координата, избираме тази с максимална y -координата сред тях.

Допускаме за противоречие, че p^* е доминирана от някоя точка $q = (x_q, y_q) \in P$, $q \neq p^*$. Тогава $x^* \leq x_q$ и $y^* \leq y_q$.

Тъй като x^* е максималната x -координата, имаме $x_q \leq x^*$, следователно $x_q = x^*$.

Тъй като y^* е максималната y -координата сред точките с x -координата x^* , имаме $y_q \leq y^*$, следователно $y_q = y^*$.

Но тогава $q = (x^*, y^*) = p^*$ — противоречие с $q \neq p^*$.

Следователно p^* е недоминирана.

Вход и изход

Вход: $X[1 \dots n]$, $Y[1 \dots n]$: масиви от рационални числа, описващи множество от точки $P = \{(X[i], Y[i]) \mid 1 \leq i \leq n\}$.

Изход: m — броят на недоминираните точки в P .

Алгоритъм

CountUndominated($X[1..n]$, $Y[1..n]$: arrays of rationals)

@1 Сортирай точките по $X[i]$ в намаляващ ред;
 при равенство по $Y[i]$ в намаляващ ред

@2 count $\leftarrow$ 0

@3 max_y $\leftarrow$ -inf

@4 for $i = 1$ to n do

 if $Y[i] > \text{max_y}$ then

 count $\leftarrow$ count + 1

 max_y $\leftarrow$ $Y[i]$

 endif

@9 endfor

@10 return count

Алчна идея: Сортираме точките по x -координата в намаляващ ред (при равни x — по y в намаляващ ред). Обхождаме точките в този ред и поддържаме максималната y -координата видяна досега. Точка $(X[i], Y[i])$ е недоминирана тогава и само тогава, когато $Y[i]$ е строго по-голяма от всички y -координати на вече обработените точки.

Доказателство за коректност

Твърдение: CountUndominated връща коректния брой недоминирани точки.

Нека след сортирането точките са $p_1, p_2, \dots, p_n$, където $p_i = (x_i, y_i)$ и $x_1 \geq x_2 \geq \dots \geq x_n$ (при равни x , y е в намаляващ ред).

Коректност ($\Rightarrow$): Ако алгоритъмът брои p_i , тя е недоминирана. Алгоритъмът брои p_i когато $y_i > \max\{y_1, \dots, y_{i-1}\}$. Допускаме за противоречие, че p_i е доминирана от $p_j = (x_j, y_j)$ за $j \neq i$, т.е. $x_i \leq x_j$ и $y_i \leq y_j$ (с поне едно строго).

Тъй като масивът е сортиран по x в намаляващ ред:

- Ако $x_j > x_i$: тогава $j < i$ (идва преди i в масива), и $y_j \geq y_i$, но тогава $y_i \leq y_j \leq \max\{y_1, \dots, y_{i-1}\}$ — противоречие с това, че $y_i > \max_y$.
- Ако $x_j = x_i$: при равенство на x сме сортирали по y в намаляващ ред, следователно ако $y_j > y_i$, тогава $j < i$ и отново $y_i < y_j \leq \max\{y_1, \dots, y_{i-1}\}$ — противоречие.
- $x_j < x_i$: невъзможно, тъй като за доминация е необходимо $x_j \geq x_i$.

Следователно p_i е недоминирана.

Коректност ($\Leftarrow$): Ако p_i е недоминирана, алгоритъмът я брои. Допускаме за противоречие, че p_i е недоминирана, но алгоритъмът не я брои. Тогава $y_i \leq \max_y = \max\{y_1, \dots, y_{i-1}\}$. Нека $j < i$ е такова, че $y_j \geq y_i$. Тъй като $j < i$, имаме $x_j \geq x_i$ (масивът е сортиран по x намаляващо). Ако $x_j > x_i$ или $y_j > y_i$, тогава p_i е доминирана от p_j — противоречие.

Времева сложност

- Ред **01**: Сортиране — $O(n \log n)$.
- Редове **04–09**: Цикъл с n итерации, всяка $O(1)$ — $O(n)$.

Следователно $\text{Time}(\text{CountUndominated}) \in O(n \log n)$.